

GO PROGRAMMING MASTER COURSE

Week 2, Day 1

Variables, types, zero values, operators, and strings — the data you will later put on a student record.

Umesh Indrajith · BSc Eng (Hons), University of Moratuwa
Senior Software Engineer / Lecturer

SINCE LAST
WEEK

We start only if the toolchain works

- go version is **1.25.x**. If not, we fix that before anyone types new code.
- Homework was why - go .md + version proof in the same GitHub repo as hello.go.
- I will ask two people: programming vs software engineering, and why Go for SMS.
- Unformatted Go is still returned. gofmt did not expire.

TODAY

If Day 1 of week 2 works, you can

- Declare values with `var`, `:=`, and `const`.
- Name the zero value of `int`, `string`, `bool`, and a pointer.
- Convert types **explicitly**. Go will not do it for you.
- Use `fmt` and `strings`; tell bytes from runes.
- Run `go test` on `FullName` — first test of the course.

Types are a design decision

A type answers: what kind of value is this, and what is allowed? That is how a team agrees before the first line of a handler.

- SMS later: student age is an `int`, email is a `string`, enrolled is a `bool`.
- Wrong type compiles only if you force it. That is a gift. Use it.
- Names: `camelCase` inside a package, `MixedCaps` to export. Comments explain *why*.

DECLARATION

var — say the name, then the type

```
var campus string = "campus"  
var seats int  
var open bool = true
```

```
var (  
    first string  
    last string  
)
```

- Works at package level and inside functions.

- If you omit the value, you still have a value: the **zero value**.

- `var seats int` is not "empty". It is 0.

DECLARATION

:= — short, only inside a function

- Declare *and* assign. Go infers the type from the right-hand side.
- Illegal at package level. Illegal if the name already exists on the left *alone* (reuse needs =).
- Inside func, this is the usual form.

```
func main() {  
    name := "Ada"  
    year := 1815  
    ok := true  
    fmt.Println(name, year)  
}
```

WHICH
ONE?

var
vs
:=

Use var

Package level. Or you want the zero value and will assign later. Or you want the type written down because the value is not obvious.

Use :=

Inside a function, first time you need the name, the value is right there. This is most of the course.

Use =

The name already exists. You are changing it, not inventing it.

Do not mix for style points

Pick one per line. gofmt will not save a confusing declaration.

CONSTANTS

const, and a light look at iota

```
const institute =  
const maxSeats = 3  
  
const (  
    statusActive =  
    statusIdle  
    statusArchived  
)
```

- A const is known at compile time. You cannot assign a variable into it.
- iota counts up in a const block. Remember the idea; do not build an enum library today.

THE TYPES
YOU WILL
ACTUALLY
USE

Start with four, then two aliases

int

Whole numbers.
Default integer. Size
follows the machine
(64-bit in this lab).

float64

Decimals. Prefer this
over float32 unless
a library forces you.

bool

true or false.
Nothing else.

string

Text in double quotes.
Immutable. UTF-8
under the hood.

byte

Alias for uint8. One
byte of data.

rune

Alias for int32. One
Unicode code point.
Written ' rune '.

THE IDEA
THAT
SAVES YOU
LATER

Zero values — Go never leaves junk

```
var n int
var s string
var ok bool
var p *int
var names []string
```

//C and some older languages give you
//garbage. Go gives you a known value. That
//is why `if s == ""` is a real check, not a
//crash.

CONVERSIONS

No implicit numeric conversion

- int and int64 are different types. So are int and float64.
- You write the conversion. The compiler will not guess.
- Truncation is your problem: int(3.9) is 3.

```
var year int = 2026
var precise float64 = f

var score float64 = 3.9
var whole int = int(score)

// year + 0.5
// int64(year) + year
```

LIVE
DEMOZERO VALUES AND
CONVERSIONS

Type this with me

```
package main
import "fmt"
func main() {
    var seats int
    var label string
    var open bool
    fmt.Printf("zeroes: %d %q %t\n", seats, la
    celsius := 33
    fmt.Println(float64(celsius) * 9 / 5 + 32)
}
```

OPERATORS

Arithmetic and assignment

```
a, b := 10, 3
sum := a + b      // 13
mod := a % b      // 1
a += 2            // 12
a++               // 13 (statement)
```

- + on strings concatenates:
"Ada" + " " + "Lovelace".
- There is no a++ + b
cleverness. ++ is a statement of
its own.
- Integer division: 10 / 3 is 3.
Use a float64 if you want
3.33.

OPERATORS

Comparison and logic

```
name := "Ada"  
fmt.Println(name == "Ada")  
fmt.Println(name != "ada")  
enrolled, paid := true, false  
fmt.Println(enrolled && paid)  
fmt.Println(enrolled || paid)  
fmt.Println(!paid)
```

■ Strings compare lexicographically, byte by byte. Good enough for homework; locale sorting is not today.

■ We will put these inside `if` next week. Today, print the `bool`.

OPERATORS
• LIGHT

Bitwise — know they exist

```
fmt.Println(1<<3) //  
fmt.Println(6 & 3) //  
fmt.Println(6 | 3) //  
fmt.Println(6 ^ 3) //
```

- You will not need these for SMS fields.
- They show up in flags, permissions, and some interview questions.
- If the lab is slow, skip the live typing here. The handout has the table.

FMT Print so another human can read it

```
fmt.Println("plain", 42)
fmt.Printf("name=%s age=%d ok=%t\n", "E", 42, true)
s := fmt.Sprintf("Hello, %s", "E")
fmt.Println(s)
```

- %s string · %d int · %f float · %t bool · %v default · %T type · %q quoted string.

- Println adds spaces and a newline. Printf does not guess — you write the verbs.

STRINGS

The package you will import every week

```
import "strings"
```

```
s := "  Ada Lovelace  "
fmt.Println(strings.TrimSpace(s))
fmt.Println(strings.ToUpper(s))
fmt.Println(strings.Contains(s, "a"))
fmt.Println(strings.Split("a,b", ","))
fmt.Println(strings.Join([]string{"a", "b"}, ","))
```

Split/Join use a slice. Copy the line; slices as a topic are week 3.

FullName homework: trim, then join with one space.

BYTES
AND
RUNES

len counts bytes, not letters

```
import "unicode/utf8"

fmt.Println(len("cafe"))
fmt.Println(len("café"))
fmt.Println(utf8.RuneCountInString("café"))
fmt.Println(len("සිංහල"))
fmt.Println(utf8.RuneCountInString("සිංහල"))
```

- A Go string is a read-only slice of bytes, usually UTF-8.
- A rune is a Unicode code point. The letter é is one rune and two bytes. සි is two runes: ස + the vowel sign.

LIVE
DEMOUTF-
8

Print what len actually measured

```
package main
import (
    "fmt"
    "unicode/utf8"
)
func main() {
    word := "ဆိမ့်မ့်"
    fmt.Printf("bytes=%d runes=%d %q\n",
        len(word), utf8.RuneCountInString(word), word)
    for i, r := range word {
        fmt.Printf("%d %c %U\n", i, r, r)
    }
}
```

TIME

Dates exist. We need them for SMS later.

```
import "time"

now := time.Now()
fmt.Println(now.Format("2006-01-02"))
fmt.Println(now.Weekday())

dob, err := time.Parse("2006-01-02", "2006-01-02")
fmt.Println(dob, err)
```

- The layout is the **reference date** Mon Jan 2 15:04:05 MST 2006 — not YYYY-MM-DD.

- Write 2006-01-02 when you want year-month-day. Memorise that pattern.

- err is a preview. If parse fails it is not nil. Next week we still will not master errors.

CLEAN
CODE
THIS
WEEK

Names and comments

camelCase

firstName, seatCount. Not
first_name in Go identifiers.

MixedCaps to export

FullName is visible from a test in
another file of the same package, and
later from other packages. fullName is
not exported.

Comments say why

Do not write `// increment i`. Write
why this number is 30, or why we trim
spaces.

gofmt still wins

Alignment, tabs, spacing — not your
taste. The review looks at names and
behaviour.

LIVE
DEMOA
TINY
CLI

Flags today, HTTP in week 5

```
package main
import ("flag"; "fmt")
func main() {
    first := flag.String("first", "", "first name")
    last := flag.String("last", "", "last name")
    flag.Parse()
    fmt.Println(*first, *last)
}
```

go run greet.go -first Ada -last Lovelace
— then put FullName in the middle. Pointers:
flag.String returns *string. Write the * to read
it. Theory is week 8.

LIVE
DEMOFIRST TEST
OF THE
COURSE

go test on a pure function

```
// names.go      package main
func FullName(first, last string) string {
    return first + " " + last
}
```

```
// names_test.go  package main
func TestFullName(t *testing.T) {
    got := FullName("Ada", "Lovelace")
    if got != "Ada Lovelace" {
        t.Errorf("Expected 'Ada Lovelace', got '%s'", got)
    }
}
```

One new command so the test can run: `go mod init epiclearn/week02`. Meaning: week 4.

Then: `go test`. Green is the gate. Improve `FullName` with `TrimSpace` for homework.

Done when all of this is true

- Worksheet: every zero-value and conversion cell filled
- `zeros.go` prints zeros and a correct Celsius → Fahrenheit
- `greet.go -first ... -last ...` prints a single full name
- `go test` in the names folder is green for `FullName("Ada", "Lovelace")`
- Commit and push to GitHub. Show me the URL before you leave

**NEXT
SESSION**

Week 3 — control flow, slices, maps

- if, switch, for, range.
- Arrays vs slices; append and the backing-array gotcha.
- Maps and the comma-ok check.
- Bring a green go test and the ten exercises.

Questions